AI-Based Smart Baby Soothing System

Chaitanyaa Dave, Kaushiki Choubey, Vedanti Dokare
K.J. Somaiya School of Engineering, Somaiya Vidyavihar University, Mumbai, Maharashtra, India, 400077

I. ABSTRACT

TwirlyTunes is an AI-based baby-soothing system designed to detect baby crying and provide an automatic soothing response through lullaby playback and gentle jhoomar rotation. Babies may cry due to hunger, discomfort, or the need for attention, requiring caregivers to respond frequently. This project aims to develop an affordable and automated system that can identify crying sounds and initiate soothing actions, helping reduce repeated manual intervention by caregivers.

The system uses a Raspberry Pi 4 as its main processing unit [5], [9] and an INMP441 digital microphone to capture surrounding audio [6]. A fine-tuned MobileNetV3-Small model [4], deployed using TensorFlow Lite [3], analyses audio samples to distinguish baby crying from non-crying sounds. Once crying is confirmed, the Raspberry Pi sends a command to an ESP32 through USB serial communication. The ESP32 [7] controls an L298N motor driver [8] to operate a DC geared BO motor, which rotates the jhoomar. Simultaneously, a pre-stored lullaby is played through a powered AUX speaker. The software, machine-learning model, and audio files are stored on a microSD card.

Frequent baby crying can require continuous attention from parents and caregivers. Manually playing soothing music or operating a mobile whenever a baby cries may become inconvenient, particularly when caregivers are occupied with other activities. Therefore, an automated system that can detect crying and initiate a suitable soothing response can provide additional convenience in everyday caregiving.

The proposed system combines audio processing, TinyML-based classification, embedded communication, and motor control to provide an integrated soothing response. A majority-voting mechanism is used to confirm crying based on consecutive model predictions, helping reduce unnecessary activations. The selected confidence threshold of 0.80 achieved a test accuracy of 95.06%, precision of 94.42%, recall of 76.89%, and an F1-score of 84.76%. These results demonstrate promising classification performance, although some actual crying samples remain undetected.

The expected outcome is a functional automated baby-soothing system capable of identifying crying sounds and activating lullaby playback and jhoomar rotation. The project demonstrates the practical application of edge AI and low-cost embedded hardware in an infant-care assistance system. However, further testing and improvements are required to enhance detection reliability, response time, and mechanical safety. TwirlyTunes is intended to serve as a supplementary soothing aid and not as a replacement for parental attention or adult supervision.

II. INTRODUCTION

The need for smart and affordable infant-care solutions is increasing as parents and caregivers often have to manage multiple responsibilities simultaneously. Babies may start crying due to hunger, discomfort, or the need for attention, and responding to them requires time and continuous supervision. A system capable of detecting a baby's cry and providing an automatic soothing response can offer additional support to caregivers in such situations. The proposed project, TwirlyTunes, aims to assist parents by combining AI-based cry detection, lullaby playback, and gentle jhoomar rotation to create a more comforting environment for babies at home or in childcare settings.

One of the major challenges addressed by this project is the need to respond whenever a baby starts crying, particularly when the caregiver is occupied with other activities. Repeatedly monitoring the baby and manually playing soothing music can become inconvenient. To address this problem, TwirlyTunes uses an INMP441 digital microphone [6] to capture surrounding audio and a TinyML-based model to identify baby-cry sounds. Once crying is confirmed, the Raspberry Pi 4 [5], [9] initiates a soothing response by playing a pre-stored lullaby through a powered AUX speaker and sending a command to an ESP32 [7], [10]. The ESP32 controls an L298N motor driver [8], which operates a DC geared BO motor attached to the jhoomar, providing gentle rotational movement alongside the music.

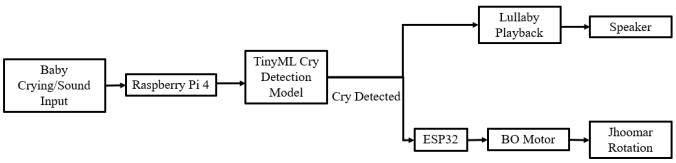


Fig. 1. System Block Diagram

Commercial baby-monitoring and soothing systems may offer features such as audio and video monitoring, automatic music playback, and smartphone notifications. However, some solutions can be expensive or unnecessarily complex for users who require basic automated soothing functions. TwirlyTunes focuses on essential features such as cry detection, automatic lullaby playback, and motorized jhoomar movement using readily available electronic components. The system uses a Raspberry Pi 4 [5], ESP32 [7], INMP441 microphone [6], L298N motor driver [8], and DC geared motor, with the electronic circuit designed using KiCad. By combining these

components with a lightweight machine-learning model, the project aims to demonstrate a practical and potentially affordable approach to automated baby soothing.

In this project, the Raspberry Pi 4 acts as the main processing unit responsible for audio capture, cry identification, and decision-making. The INMP441 microphone captures audio samples, which are processed into Mel-spectrograms and analysed using a fine-tuned MobileNetV3-Small model [4] deployed through TensorFlow Lite [3]. A majority-voting mechanism is used to confirm crying and reduce unnecessary activations. Once crying is confirmed, the Raspberry Pi plays a pre-stored lullaby through the powered AUX speaker and communicates with the ESP32 through USB serial communication [10]. The ESP32 generates the required motor-control signals through the L298N driver [8] to rotate the DC geared BO motor connected to the jhoomar. This combination of AI-based sound detection, soothing music, and gentle mechanical movement is intended to provide an automated response that may help comfort the baby while reducing repeated manual intervention by caregivers.

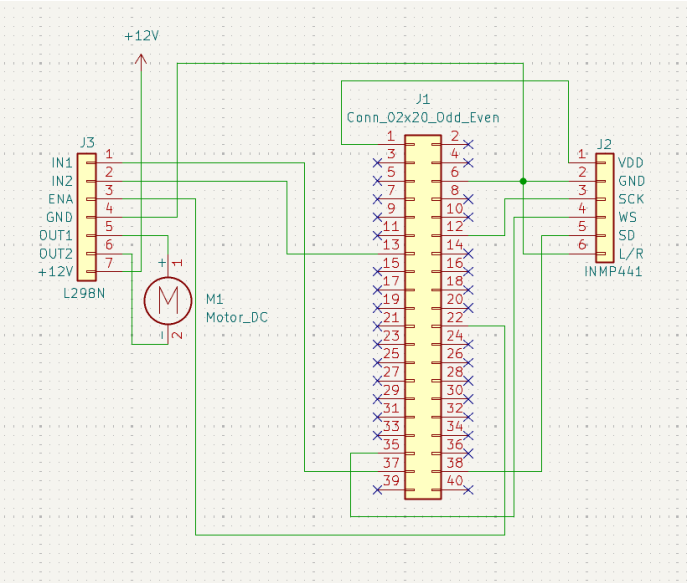


Fig. 2. Circuit Schematic

III. Literature Review

1. Infant Cry Analysis and Classification

Baby crying is an important form of communication through which infants express their needs and discomfort. Traditional monitoring mainly depends on caregivers hearing and interpreting the baby's cry. Ji et al. (2021), in their research paper A Review of Infant Cry Analysis and Classification, reviewed different approaches to infant-cry analysis using audio processing and machine-learning techniques. The study discussed features such as Mel-Frequency Cepstral Coefficients (MFCC), spectrograms, and fundamental frequency, along with classifiers such as K-Nearest Neighbours (KNN), Support Vector Machines (SVM), and neural networks. These techniques demonstrate how audio signals can be analysed to distinguish infant crying from other sounds. This research provides a foundation for using the INMP441 microphone and Mel-spectrogram-based audio processing in the proposed TwirlyTunes system.

2. Machine Learning for Infant Cry Detection

Machine learning and deep-learning techniques have made it possible to identify patterns in audio recordings and classify different types of sounds automatically. Owino and Shibwabo (2025), in their systematic review Advances in Infant Cry Paralinguistic Classification—Methods, Implementation, and Applications, examined various approaches used for infant-cry classification. The review highlighted the use of audio features such as MFCCs, spectrograms, pitch, duration, and intensity, along with different machine-learning methods. It also discussed the challenges associated with limited datasets, background noise, and variations in crying patterns. These findings support the use of a trained deep-learning model for baby-cry detection in TwirlyTunes. Accordingly, the project uses a fine-tuned MobileNetV3-Small model [4] to classify audio into two categories: non-cry and baby-cry.

3. TinyML and Edge-Based Audio Classification

TinyML and edge-based machine learning focus on performing model inference directly on local computing devices instead of depending entirely on cloud-based processing. TensorFlow's audio-classification example for Raspberry Pi demonstrates how microphone input can be processed using a TensorFlow Lite model to recognise different sounds [3], [9]. This approach supports local audio analysis and allows classification to take place on the device. In TwirlyTunes, the Raspberry Pi 4 acts as the main processing unit and runs the fine-tuned MobileNetV3-Small model using TensorFlow Lite. Audio captured by the INMP441 microphone is divided into short windows, converted into Mel-spectrograms, and supplied to the model for classification. This implementation demonstrates the practical use of lightweight deep learning for local audio recognition without requiring continuous cloud communication.

4. Automated Baby-Soothing Systems

Automated baby-soothing systems aim to respond to infant crying by providing comforting stimuli such as lullabies and gentle rocking movements. A 2025 research paper titled A Novel Smart Baby Cradle System Utilizing IoT Sensors and Machine Learning for Optimized Parental Care explored the integration of machine learning, infant monitoring, automated rocking, and lullaby playback. The system demonstrates how cry recognition can be combined with physical and audio-based soothing mechanisms to support caregivers. This research is closely related to the concept of TwirlyTunes, which combines automatic lullaby playback with motorized jhoomar rotation. However, TwirlyTunes uses a simpler hardware arrangement consisting of a Raspberry Pi 4, ESP32, L298N motor driver, and DC geared BO motor. Its purpose is to demonstrate an integrated and potentially affordable soothing solution using commonly available components.

5. Research Gap and Proposed System

Existing research demonstrates the potential of audio processing and machine learning for identifying infant cries, while automated cradle projects show how crying detection can be connected to soothing mechanisms. However, there is still scope for developing simple and affordable systems that combine cry detection with automated audio playback and mechanical movement. TwirlyTunes addresses this area by integrating a fine-tuned MobileNetV3-Small model with embedded hardware to provide an automatic soothing response. The INMP441 microphone captures audio, the Raspberry Pi 4 performs cry detection, and the ESP32 communicates with the Raspberry Pi to control the L298N motor driver and BO motor. At the same time, a pre-stored lullaby is played through a powered AUX speaker. The system is evaluated using classification metrics such as accuracy, precision, recall, and F1-score, along with practical testing of its audio and motor functions. Its contribution is the integration of these functions into a single system rather than the development of a new cry-classification algorithm.

IV. Results

The machine-learning component of TwirlyTunes was evaluated as a binary audio-classification model with two classes: *Baby Cry* and *Non-Cry*. The model uses a fine-tuned MobileNetV3-Small architecture to classify audio represented as log-Mel spectrograms. The spectrograms were resized to $224 \times 224 \times 3$ to match the expected model input dimensions. The training dataset combined baby-cry recordings from Donate-a-Cry [2], environmental sounds from ESC-50 [1], and real-world non-cry recordings. The reconstructed dataset consisted of 7,960 audio windows, divided into 4,962 training windows, 1,519 validation windows, and 1,479 test windows.

The final model was evaluated on the held-out test set using a decision threshold of 0.80. The test set contained 1,215 Non-Cry samples and 264 Baby Cry samples. The evaluation considered accuracy, precision, recall, F1-score, ROC-AUC, and average precision to assess the model's classification performance.

A. Classification Performance

The final model achieved a test accuracy of 95.06%, with a precision of 94.42%, recall of 76.89%, and F1-score of 84.76%. The ROC-AUC was 0.9802, while the average precision was 0.9391. Table I presents the overall classification metrics obtained on the held-out test set.

The precision and recall values provide complementary information about the model's performance. The precision of 94.42% indicates that a high proportion of the samples predicted as Baby Cry were correctly classified. The recall of 76.89% indicates the proportion of actual baby-cry samples successfully identified by the model. The F1-score of 84.76% provides a combined measure of precision and recall.

B. Confusion Matrix Analysis

The confusion matrix provides a detailed representation of the model's predictions for both classes. Table II presents the results obtained using the selected decision threshold of 0.80.

TABLE I
Classification Performance on the Held-Out Test Set

Evaluation Metric	Value
Test samples	1,479
Test accuracy	95.06%
Precision	94.42%
Recall	76.89%
F1-score	84.76%
ROC-AUC	0.9802
Average precision	0.9391
Decision threshold	0.80

TABLE II
Confusion Matrix for the Held-Out Test Set

Actual / Predicted	Non-Cry	Baby Cry
Non-Cry	1203	12
Baby Cry	61	203

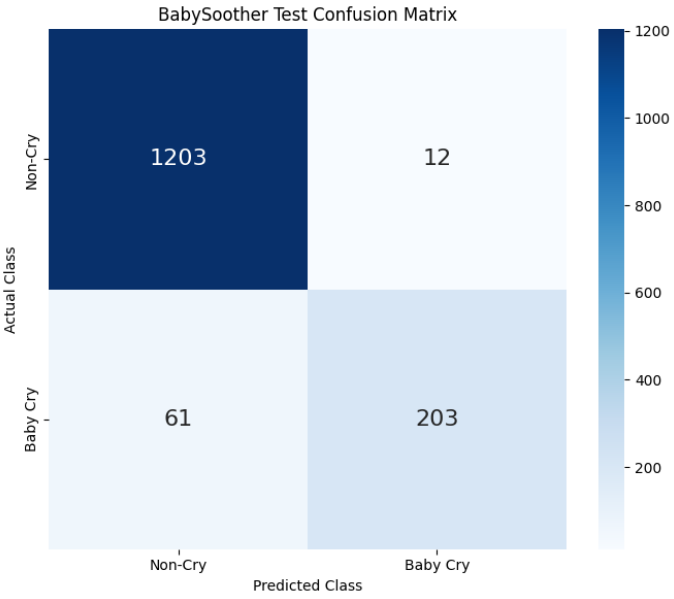


Fig. 3. Confusion Matrix

Out of 1,215 Non-Cry samples, 1,203 were correctly classified, while 12 were incorrectly predicted as Baby Cry. Similarly, out of 264 Baby Cry samples, 203 were correctly identified and 61 were classified as Non-Cry. Thus, the model correctly classified 1,406 out of 1,479 test samples.

The accuracy is calculated as follows:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \times 100 \qquad (1)$$

$$\text{Accuracy} = \frac{203 + 1203}{203 + 1203 + 12 + 61} \times 100 = 95.06\%. \qquad (2)$$

C. ROC and Precision–Recall Analysis

The Receiver Operating Characteristic (ROC) curve was used to evaluate the model's ability to distinguish between Baby Cry and Non-Cry samples across different classification thresholds. The ROC curve plots the True Positive Rate against

the False Positive Rate. The model achieved an Area Under the ROC Curve (ROC-AUC) of 0.9802, indicating strong class-discrimination performance across the evaluated thresholds.

The Precision–Recall curve provides an additional assessment of the relationship between precision and recall at different thresholds. The average precision obtained was 0.9391. These metrics provide a threshold-independent view of model performance and complement the confusion matrix obtained at the selected operating threshold.

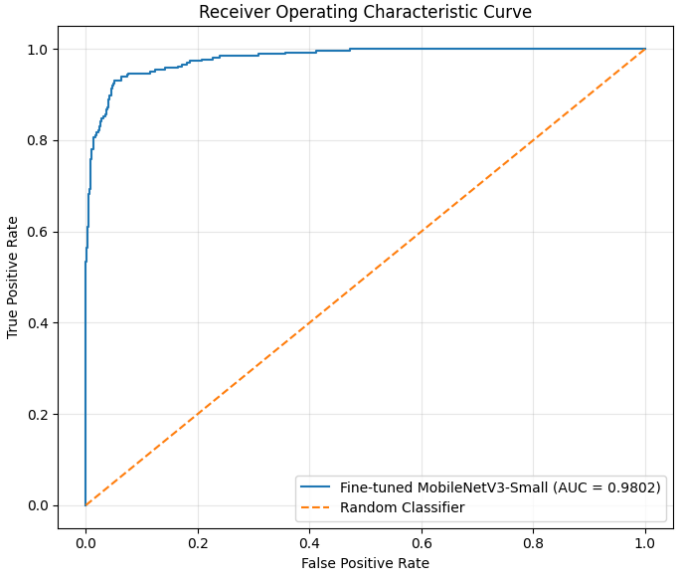


Fig. 4. ROC Curve

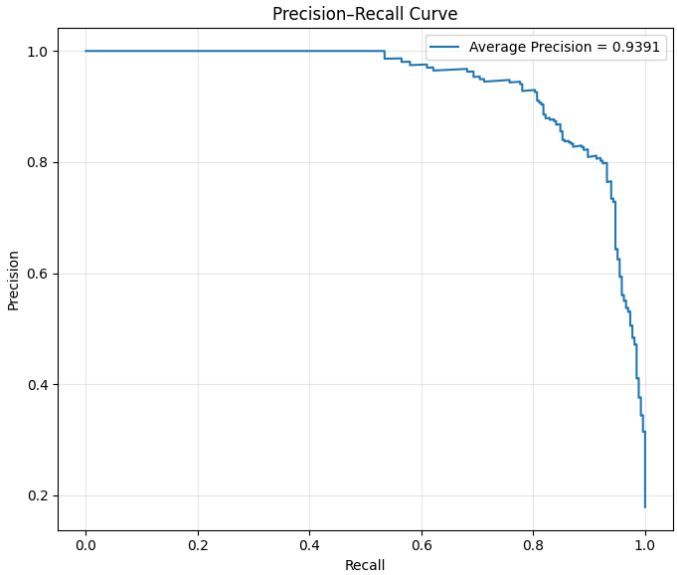


Fig. 5. Precision Recall Curve

D. Threshold Analysis

The decision threshold determines the minimum sigmoid probability required for an audio window to be classified as Baby Cry. A lower threshold increases the sensitivity of the classifier but may also increase false-positive predictions. Conversely, a higher threshold makes the classifier more selective and can reduce false positives, but may result in additional missed crying samples.

After evaluating different thresholds, a value of 0.80 was selected for the final system configuration. At this threshold, the model achieved 95.06% accuracy and 94.42% precision on the held-out test set, with a recall of 76.89%. This threshold was used as the operating point for the subsequent system-level detection logic.

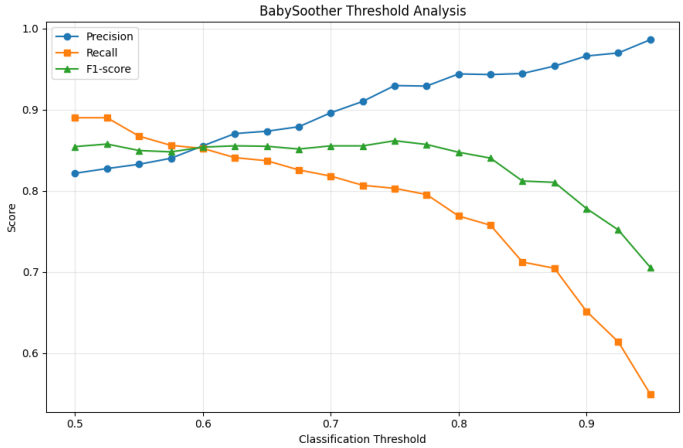


Fig. 6. Threshold Analysis Graph

E. False-Positive and False-Negative Analysis

Further error analysis was performed to identify the audio categories responsible for incorrect predictions. The false-positive analysis identified 12 Non-Cry samples that were classified as Baby Cry. These included samples associated with cat sounds, toilet flushing, hens, dogs, laughter, roosters, trains, and real-world non-cry recordings.

The false-negative analysis identified 61 Baby Cry samples that were classified as Non-Cry. Most of these missed samples belonged to the hungry category, followed by belly-pain and discomfort categories. These categories were used for error analysis and were not separate output classes of the binary classifier.

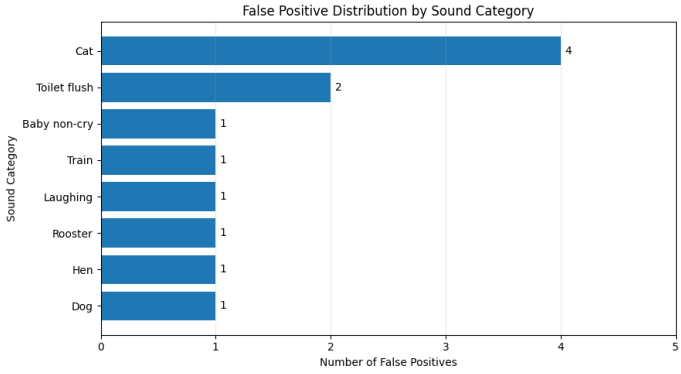


Fig. 7. False-Positive Distribution

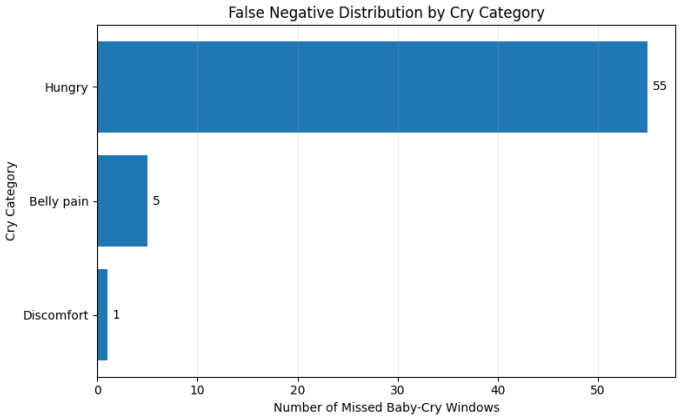


Fig. 8. False-Negative Distribution

F. TensorFlow Lite Conversion and Deployment Configuration

Following training and fine-tuning, the model was converted into TensorFlow Lite format for deployment on the Raspberry Pi 4. The TensorFlow Lite interpreter reported an input tensor shape of $1 \times 224 \times 224 \times 3$ and an output tensor shape of 1×1, consistent with the expected input and sigmoid probability output of the binary classifier.

The converted TensorFlow Lite model was compared with the original Keras model using the same input. The maximum observed difference between their predicted probabilities was approximately 7.41×10^{-7}, demonstrating close numerical agreement between the two model formats for the tested input.

The final system configuration uses an individual prediction threshold of 0.80 and a majority-voting mechanism that requires at least six positive predictions among the latest ten predictions before confirming crying. Following confirmation, the Raspberry Pi initiates approximately 20 seconds of lullaby playback and sends a control command to the ESP32 for jhoomar rotation.

G. Hardware and System Integration

The hardware architecture integrates the Raspberry Pi 4, INMP441 microphone, ESP32 microcontroller, L298N motor driver, DC motor, and powered AUX speaker. The Raspberry Pi performs audio acquisition, preprocessing, model inference, and overall system coordination. The ESP32 receives motor-control commands through USB serial communication [10] and controls the motor driver [8].

The motor operation uses a PWM profile that starts at 85 and gradually decreases to 60 over approximately 1.5 seconds. This gradual change is intended to provide smoother rotation of the jhoomar. The soothing sequence is configured for approximately 20 seconds, after which the playback and motor operation are stopped and audio monitoring resumes.

The model evaluation results are reported separately from the hardware response. The reported classification metrics represent held-out audio-window performance and should not be interpreted as measurements of continuous real-world detection reliability or end-to-end response time.



Fig. 9. Final Product

V. Discussion

The experimental results demonstrate the potential of MobileNetV3-Small for lightweight baby-cry classification using log-Mel spectrogram representations. The final model achieved 95.06% accuracy and a ROC-AUC of 0.9802 on the held-out test set. These results indicate that the model learned useful time-frequency patterns for distinguishing baby-cry recordings from non-cry audio samples.

A. Interpretation of Classification Performance

The model achieved a precision of 94.42%, indicating that most audio windows classified as Baby Cry were correctly labelled. This is important for the proposed application because false-positive predictions can lead to unnecessary activation of the soothing mechanisms.

However, the recall of 76.89% indicates that the model did not identify all actual baby-cry windows. The confusion matrix shows 61 false-negative predictions compared with 12 false-positive predictions. This difference suggests that, at the selected threshold of 0.80, the classifier is relatively selective when identifying crying samples.

In an infant-soothing application, false negatives require particular attention because they represent crying audio that the system fails to recognize. Therefore, improving recall while maintaining an acceptable level of precision is an important objective for future model development. The F1-score of 84.76% provides a combined measure of these two performance characteristics.

B. Effect of Decision Threshold

The decision threshold has a direct influence on the behaviour of the classifier. A lower threshold allows more audio windows to be classified as Baby Cry, potentially improving recall while increasing the number of false-positive predictions. A higher threshold reduces the number of positive predictions but may cause quieter or acoustically different crying samples to be missed.

The threshold of 0.80 was selected after evaluating different operating thresholds and was used for the final implementation. Although this configuration achieved high precision, the lower recall indicates that further threshold optimization may be beneficial. The selection of a final operating threshold should consider the relative importance of missed crying events and unnecessary soothing activations, rather than relying on accuracy alone.

C. Analysis of Classification Errors

The false-positive analysis identified environmental sounds such as cat sounds, toilet flushing, hens, dogs, laughter, roosters, and trains as sources of incorrect predictions. Some of these sounds may contain frequency and temporal patterns that overlap with characteristics learned by the model from baby-cry recordings. This highlights the importance of including diverse environmental sounds and challenging negative examples during training.

The false-negative analysis showed that most missed samples belonged to the hungry category, followed by belly-pain and discomfort categories. Different crying episodes can have variations in duration, pitch, intensity, and frequency distribution. Consequently, the model may not recognize every crying pattern equally well, particularly when certain patterns are insufficiently represented in the training data.

These observations suggest that expanding the dataset with more diverse baby-cry recordings and challenging non-cry examples could improve the model's generalization. Further analysis of missed samples may also help identify specific acoustic patterns that require additional training examples.

D. Role of Majority Voting

In addition to the individual model decision threshold, TwirlyTunes uses a majority-voting mechanism to improve the stability of the overall detection process. The system considers the latest ten predictions and confirms crying when at least six predictions are positive.

This mechanism reduces the influence of isolated positive predictions that may result from temporary background noise or other non-cry sounds. By considering multiple consecutive predictions, the system aims to avoid unnecessary activation of the lullaby and motor mechanisms.

However, majority voting does not eliminate all classification errors. Consecutive incorrect predictions may still satisfy the voting condition, while intermittent or short-duration crying events may not produce enough positive predictions to trigger the system. The mechanism also introduces a delay that depends on the rate at which predictions are generated. Its effectiveness should therefore be evaluated using continuous audio recordings and system-level testing.

E. Suitability of MobileNetV3-Small for Embedded Deployment

MobileNetV3-Small was selected because it is designed for computational efficiency and is suitable for applications with limited hardware resources [4]. Its use with log-Mel spectrograms allows the audio classification task to be handled using a lightweight convolutional neural network.

Converting the trained model into TensorFlow Lite format provides a deployment-oriented representation for inference on the Raspberry Pi 4 [3], [5], [9]. The close numerical agreement observed between the Keras and TensorFlow Lite predictions for the tested input supports the correctness of the conversion process for that comparison.

Nevertheless, model conversion alone does not establish real-time performance. Further measurements of inference latency, processor utilization, memory consumption, and sustained operation on the Raspberry Pi 4 are required to quantify deployment efficiency.

F. Hardware Integration and System-Level Considerations

The proposed system combines audio-based detection with two soothing mechanisms: lullaby playback and jhoomar rotation. The Raspberry Pi 4 performs the computational tasks and coordinates the system, while the ESP32 handles motor-control operations through the L298N motor driver.

The use of a separate motor-control unit reduces the need for the Raspberry Pi to generate motor-control signals directly. The PWM ramp, which decreases from 85 to 60 over approximately 1.5 seconds, is intended to provide a gradual change in motor operation.

The system is configured to play the lullaby and rotate the jhoomar for approximately 20 seconds after crying is confirmed. However, classification performance and physical system performance are separate evaluation aspects. The model's test accuracy does not directly establish the reliability of the complete hardware system. Continuous operation, serial communication reliability, audio playback behaviour, motor response, and end-to-end response time should be measured independently.

G. Limitations and Future Improvements

Despite the encouraging classification results, several limitations remain. The performance of the model depends on the diversity and quality of the training dataset, microphone placement, and surrounding acoustic conditions. Real-world

environments may contain overlapping speech, television audio, music, reverberation, and other sounds that differ from the samples available during training.

The current classifier performs binary classification and does not identify the specific reason behind an infant's crying. The soothing response is predefined and does not adapt to individual crying patterns or the baby's response to the selected lullaby and jhoomar movement.

Future work can focus on expanding the training dataset, collecting more real-world recordings, improving the detection of underrepresented crying patterns, and evaluating alternative model architectures. Multi-class cry classification may also be explored to distinguish between different categories of crying, provided that sufficiently labelled data are available.

At the system level, future improvements may include remote monitoring, additional soothing mechanisms, adaptive response control, and systematic testing under different environmental conditions. Quantitative evaluation of inference latency, power consumption, communication reliability, and continuous operating performance would provide a more complete assessment of the prototype.

H. Overall Discussion

Overall, the results indicate that the fine-tuned MobileNetV3-Small model provides a promising foundation for audio-based baby-cry classification in TwirlyTunes. The integration of the model with the Raspberry Pi 4, ESP32, motor driver, DC motor, and speaker demonstrates the proposed combination of machine learning and embedded hardware.

The model evaluation establishes its classification performance on the held-out test dataset, while the majority-voting mechanism and predefined soothing sequence provide the logic for automated system operation. Further real-world and system-level evaluation is necessary before making claims about continuous monitoring reliability or practical effectiveness in infant-care environments. TwirlyTunes is therefore intended as an assistive prototype and not as a replacement for parental supervision or professional infant care.

VI. Conclusion

The developed TwirlyTunes system demonstrates the integration of machine learning and embedded hardware for an automated baby-soothing application. The system uses a Raspberry Pi 4 as the central processing unit, an INMP441 microphone for audio acquisition, and a fine-tuned MobileNetV3-Small model deployed using TensorFlow Lite for baby-cry detection. Audio recordings are converted into log-Mel spectrograms and processed by the model to distinguish between Baby Cry and Non-Cry sounds. The final model achieved an accuracy of 95.06%, with a precision of 94.42%, recall of 76.89%, and F1-score of 84.76% on the held-out test dataset.

Upon confirmation of crying through a majority-voting mechanism, which requires at least six positive predictions among the latest ten predictions, the Raspberry Pi initiates the soothing process. This involves playing a predefined lullaby through a powered AUX speaker and communicating with an ESP32 to operate a DC motor through an L298N motor driver, thereby rotating the jhoomar. The system integrates audio acquisition, machine-learning-based classification, audio playback, and motor control into a unified prototype, reducing the need for caregivers to manually activate these soothing mechanisms.

The development of TwirlyTunes also provided practical experience in audio preprocessing, transfer learning, model optimization for embedded deployment, circuit design, hardware integration, serial communication, and motor control. The results demonstrate the potential of lightweight deep-learning models for audio classification on resource-constrained platforms. However, the current model performs binary classification and does not identify the specific reason behind an infant's crying. Furthermore, the reported classification metrics represent held-out audio-window performance and do not independently establish the reliability or effectiveness of the complete system in real-world infant-care environments.

Future improvements may include expanding the training dataset, improving the detection of diverse crying patterns, introducing multi-class cry classification, incorporating remote notifications and manual controls, and optimizing power consumption. Further testing under different environmental conditions is also necessary to evaluate inference latency, continuous detection performance, communication reliability, and the physical response of the soothing mechanisms. Overall, TwirlyTunes demonstrates a promising approach to combining TinyML and embedded systems for automated soothing. It is intended only as a supplementary assistive solution under adult supervision and must not replace direct parental care or professional infant monitoring.

VII. References

[1] K. J. Piczak, "ESC: Dataset for Environmental Sound Classification," in *Proc. 23rd ACM Int. Conf. Multimedia*, 2015, pp. 1015–1018.

[2] Donate-a-Cry Corpus, GitHub Repository. [Online]. Available: https://github.com/gveres/donateacry-corpus

[3] TensorFlow, "TensorFlow Lite Documentation." [Online]. Available: https://www.tensorflow.org/lite

[4] A. Howard *et al.*, "Searching for MobileNetV3," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2019, pp. 1314–1324.

[5] Raspberry Pi Ltd., "Raspberry Pi 4 Model B Datasheet." [Online]. Available: https://datasheets.raspberrypi.com/rpi4/raspberry-pi-4-datasheet.pdf

[6] TDK InvenSense, "INMP441 Omnidirectional Microphone Datasheet." [Online]. Available: https://invensense.tdk.com/wp-content/uploads/2015/02/INMP441.pdf

[7] Espressif Systems, "ESP32 Series Datasheet." [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

[8] STMicroelectronics, "L298 Dual Full-Bridge Driver Datasheet." [Online]. Available: https://www.st.com/resource/en/datasheet/l298.pdf

[9] Raspberry Pi Ltd., "Raspberry Pi Hardware Documentation." [Online]. Available: https://www.raspberrypi.com/documentation/computers/raspberry-pi.html

[10] Espressif Systems, "ESP-IDF Programming Guide: UART Communication." [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/uart.html